University of Oxford

Robustness of In-Context Learning in Transformers: Linear vs Nonlinear Attention under Adversarial Prompt Perturbations

by

Shujun Zhao

Lady Margaret Hall

A dissertation submitted in partial fulfilment of the degree of Master of Science in Applied Statistics.

Department of Statistics, 24–29 St Giles,
Oxford, OX1 3LB

September 2025

This is my own work (except where otherwise indicated)

Candidate: Shujun Zhao

Signed:.....

Date:.....

Abstract

Transformers exhibit in-context learning, yet their robustness to prompt-time perturbations remains poorly understood. Existing studies largely target large language models or simplified settings, rarely isolating the contribution of the attention mechanism. We address this gap by comparing linear and softmax attention-only Transformers in a controlled binary classification regime. We introduce a paired task suite (linear and parabolic), unify masking, readout and normalisation, and design simple geometrical attacks at the demonstration level: label flipping and label hijacking. We adapt robustness score and mean corruption error to the ICL setting with a GPT-2 reference. Empirically, under label flipping, linear attention is consistently more robust; under label hijacking, one-layer models behave similarly, while depth improves softmax on the parabolic task and linear remains marginally ahead on the linear task. Results indicate that robustness is jointly governed by attention rule, task geometry and depth, offering a clean baseline for future work.

Acknowledgements

Firstly, I would like to express my sincere gratitude to my supervisors, Patrick Rebeschini and Xiaoqi (Shirley) Liu, for their invaluable guidance, support, and encouragement throughout this project. It has been a privilege to carry out my first piece of research under their supervision. I am also grateful to my friends, Liangxiao Li and Nicholas Roy, for their generous help and many helpful discussions during the preparation of this work.

Contents

List of Figures

List of Tables

1 Introduction

Transformer-based models [1] have demonstrated strong performance across natural language processing (NLP) tasks, including machine translation and text classification. Scaling model capacity and data diversity has given rise to large language models (LLMs), exemplified by GPT-3 [2], which exhibit broad, general-purpose capabilities [3]. A particularly intriguing phenomenon is *in-context learning* (ICL): when presented with a prompt comprising a short sequence of input–output pairs (commonly referred to as *demonstrations*), together with a *query* from the same task, the model can produce an appropriate prediction without any parameter updates. Figure 1 illustrates this setup. This emergent ability [4] underscores the potential of LLMs as tools for tackling increasingly complex problems, including arithmetic reasoning [5, 6], code generation [7], and data engineering [8, 9].

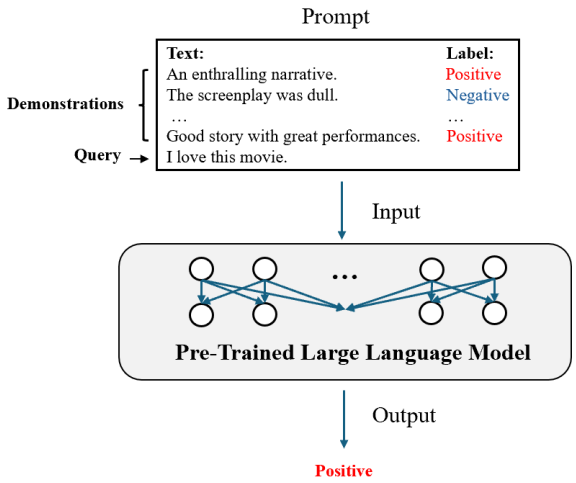


Figure 1: Illustration of in-context learning for a sentiment classification task. Given n labelled demonstrations and an unlabelled query, a pre-trained large language model predicts the query label without parameter updates.

Despite its promise, ICL is not uniformly reliable. Recent empirical studies indicate that performance is highly sensitive to factors at both pre-training and inference stages: during pre-training, domain diversity [10, 11] and distributional properties such as burstiness in the training data [12] shape the emergence and strength of ICL; during inference, prediction outcomes depend on the ordering of demonstrations and on distributional mismatch between demonstrations and queries [13, 14]. Other determinants of ICL ability include model capacity, the number of few-shot demonstrations and architectural choices [2, 15]. Of particular concern is ICL-enabled prompt manipulation in LLMs, which can elicit false or harmful responses [16]. Taken together, these observations point to the central role of output stability in ICL, an application-level notion closely tied to *robustness* [17].

1.1 Motivation

One definition of robustness in machine learning is the capacity of a model to maintain stable predictive performance when the input data vary [18, 17]. A substantial body of work examines robustness in ICL, predominantly with LLMs, offering insights into real-world behaviours [19, 20]. Two examples are illustrative. First, *jailbreaking attacks* inject malicious instructions or demonstrations into the prompt [21]; for instance, including a growing number of forbidden question–answer pairs can circumvent safety defences and elicit harmful replies, such as instructions on "how to build a bomb" [22]. Second, *hijacking attacks* use factually correct but strategically misleading demonstrations to steer predictions towards attacker-specified outputs [23, 24]. These fall under *adversarial robustness*, namely robustness to distributional shifts intentionally induced to deceive or mislead the model [17], thereby posing substantial risks to reliability and safety issues.

Understanding what drives robustness directly in LLMs and broader NLP settings is challenging. In practice, corpora are large, noisy, and difficult to characterise, while modern architectures comprise many interacting components. Encouragingly, recent work shows that Transformer architectures can implement in-context learning for function classes [25]. Building on this line, ICL is commonly viewed as learning to infer a target function $f \in \mathcal{F}$ from a few demonstrations, a viewpoint that has yielded both empirical and theoretical insights [26, 27, 28]. Concretely, a task is modelled as $f : \mathcal{X} \to \mathcal{Y}$; the prompt comprises demonstrations $\{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^{n}$ and an unlabelled query $\mathbf{x}_{\text{query}}$, where all inputs are i.i.d. from $\mathcal{P}_{\mathcal{X}}$, and the objective is to predict $f(\mathbf{x}_{\text{query}})$. Within this setup, robustness can be quantified by measuring predictive accuracy under prompt-time perturbations.

Recent work investigates the robustness of ICL in Transformers on tasks such as linear regression [29] and linear classification [30, 31], since these simplified tasks are more amenable to theoretical analysis. Much of this literature follows a common pipeline: either developing theory for linear attention–only Transformers or providing empirical comparisons using GPT-2–style architectures. Starting from linear attention has some advantages. Computationally, linear attention has been proposed to break the quadratic barrier by replacing softmax attention with a kernel feature-map formulation, yielding linear-time complexity in sequence length [32]. Moreover, many studies have shown in supervised settings such as linear regression, a single-layer Transformer with linear attention can implement gradient-descent–based updates on the in-context data [26, 27]. This modelling choice provides cleaner, controlled comparisons and experiment speedups when varying the number and length of in-context demonstrations. While developing complete theory for GPT-2–style models is hard, jumping directly from linear attention to GPT-2 may introduce too many architectural factors. Few studies cleanly isolate the contribution of the non-linear, softmax-based attention layer to robustness. Therefore, clarifying when and why attention mechanisms confer robustness requires controlled architectures.

1.2 Main Contributions

In this thesis, we directly investigate how non-linear attention affects robustness through our proposed experimental and model designs. We focus our analysis on adversarial robustness. Unless stated otherwise, "robustness" refers to adversarial robustness through-

out this thesis. We compare *linear attention-only* and *softmax attention-only* Transformers, abstracting away components that could confound ICL robustness analysis. Most prior studies consider simple regression tasks [25, 26, 33, 34, 35]. However, for continuous targets, adversarial evaluation typically relies on gradient-based sample generation via backpropagation [29], which couples the attack to model internals and complicates attribution. By contrast, we focus on *binary classification* in ICL [31, 30]. This choice offers three benefits: (i) it enables interpretable, label-space attacks that mirror jailbreaking and context hijacking; (ii) our specific problem setup permits straightforward, model-agnostic, problem-oriented attacks without backpropagation; and (iii) it allows simple design of richer input distributions and highlights the contrast between linear and non-linear attention. Motivated by previous attack patterns, we consider two prompt-time adversaries: *label flipping* and *label hijacking* via manipulation of demonstrations. As summary metrics, we report the *robustness score* [36] and *mean Corruption Error (MCE)* [37], using a GPT-2–style model as the reference. We summarise the contributions as follows:

- We adopt a standard *linear classification* task and introduce a *parabola classification* task. The parabola task increases difficulty relative to the linear case, with a tunable curvature parameter. This paired design also probes inductive bias under increasing depth: empirically, as layers increase, softmax attention gradually relaxes its bias toward linear decision rules and better accommodates the non-linear parabola task, whereas linear attention remains biased and does not exhibit the same improvement.
- We make a fully isolated comparison of attention mechanisms, linear attention-only versus softmax attention-only, removing stack-level confounds (multi-head attention, MLP blocks, depth). We train one- to four-layer attention-only Transformers and find that both mechanisms generalise when the test-time demonstration length is shorter than in training. This first confirms that our problem formulation and attention-only architectures are well-posed for ICL. Consequently, we establish a clean, non-degenerate baseline on which robustness can be meaningfully assessed.
- We design two prompt-time adversaries inspired by practical LLM risks (jailbreaking and context hijacking), implemented at the demonstration level. The two attacks respectively simulate settings with fully incorrect demonstrations and ambiguous demonstrations.
- We adapt two quantitative robustness metrics to the ICL setup: the robustness score (RS) and mean Corruption Error (mCE), enabling analysis under varying levels of demonstration contamination. The latter has been specifically modified to include a comparison with a GPT-2 baseline. Specifically, the robustness score quantifies performance under a given attack type at a specified contamination level, whereas mCE aggregates error across attack types and severities, providing an overall summary of robustness.
- We evaluate on held-out test data and report results separately for positive and negative queries. For one-layer models, the linear and softmax variants behave similarly on both tasks, indicating comparable decision rules. With increased depth, differences emerge. Under label flipping on the linear task, softmax is less robust than

linear at matched depth. Additional layers mainly flatten the decline and do not remove the gap. Under label hijacking, robustness remains high for all models. On the parabolic task, deeper softmax maintains both positive and negative accuracy and slightly exceeds linear, whereas on the linear task linear attention remains marginally ahead.

1.3 Related Work

In-Context Learning in Transformers. Traditional machine learning typically requires substantial labelled data and explicit parameter updates during training. To mitigate data sparsity, meta-learning ("learning to learn") and multi-task learning train models with shared backbones and jointly optimised objectives [38, 39, 40], though their effectiveness depends on carefully curated, related tasks and shared inductive biases. In NLP, early systems such as GPT-1 and BERT [41, 42] popularised pre-training followed by fine-tuning, coupling generative pre-training on large unlabelled corpora with task-specific supervision. As model scale and data diversity increased, large language models began to exhibit task-agnostic competence, including zero-shot [43] and few-shot [2] generalisation via prompts (i.e., ICL). From a transfer-learning perspective [44], ICL constitutes a distinct paradigm in which models adapt to new tasks at inference time by conditioning on a small set of in-prompt demonstrations, without parameter updates. Since contemporary LLMs are built on Transformer architectures, ICL is widely attributed, at least in part, to properties of attention: Transformers parallelise computation and capture long-range dependencies [1], becoming the default backbone in NLP [43, 42, 2] and extending successfully to vision via Vision Transformers (ViTs) [45].

The mechanisms underlying ICL in Transformers have been investigated from multiple angles. Under distributional assumptions, work ranging from Hidden Markov Models (HMMs) [46] to general latent-variable models [47] views ICL as Bayesian inference over latent semantics, updating a posterior given demonstrations. From mechanistic interpretability, multi-layer Transformers have been shown to develop *induction heads* that copy and continue patterns present in the prompt [48]. On the theoretical side, results on expressivity demonstrate that Transformers can represent broad function classes relevant to ICL [28]. However, most studies emphasise the capability of ICL. Comparatively little work examines its robustness to prompt-time perturbations, and even fewer studies identify whether any robustness arises from the attention mechanism itself.

Softmax vs. Linear attention. Beyond the linear attention–only results noted above, non-linear attention–only architectures have also been explored, and they appear to support more flexible in-context procedures. For example, Bai et al. [49] show that Transformers can perform *in-context algorithm selection*, adaptively choosing among base learners and hyperparameters from the prompt. Huang et al. [35] provide an early analysis of softmax attention on linear regression, establishing convergence guarantees even under non-uniform data. This observation raises the question of whether the additional flexibility afforded by softmax attention incurs a robustness cost or, conversely, inherits the stronger robustness often observed in large softmax-based models.

Adversarial robustness. Adversarial robustness matters because modern learning systems can be highly sensitive to small, structured perturbations that often transfer across models and operating conditions [50, 51]. Two complementary threat models are typically considered: test-time evasion, where inputs are crafted to induce errors [52, 53], and training-time poisoning, where features and labels are manipulated to skew the learned decision rule [54]. These concerns naturally project to ICL, in which the prompt becomes the attack surface [21, 23, 24, 55]. Studies in simplified ICL settings provide tractable ideas for theory [29, 30, 31]. Understanding how to attack clarifies failure modes and, crucially, informs the design of effective defences for ICL.

2 Preliminaries

This section formalises the framework for in-context learning used in our analysis. We first define a binary classification task under two data-generating regimes, and then specify the attention-only Transformer model used to perform ICL on this task.

Notation and conventions. Throughout, vectors are column vectors. The symbols $\mathbf{0}_d$ and $\mathbf{1}_n$ denote the all-zeros vector in $\mathbb{R}^d$ and the all-ones vector in $\mathbb{R}^n$. Boldface denotes vectors and matrices (e.g., $\mathbf{x}$, $\mathbf{A}$), while scalars are set in italic (e.g., y, k). Let d denote the feature dimension and $\mathbf{I}_d$ denote the $d \times d$ identity matrix. For a matrix $\mathbf{M}$, $[\mathbf{M}]_{ij}$ denotes its (i, j)-entry. Denote $\mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$ as the standard normal distribution on $\mathbb{R}^d$. We write $\mathbb{P}(\cdot)$ for probability and $\mathbb{E}[\cdot]$ for expectation; when needed, we use $\mathbb{E}_{X\sim\mathcal{D}}[\cdot]$ to indicate the underlying law $\mathcal{D}$ and $\mathbb{E}[\cdot \mid \cdot]$ for conditional expectation. We write $\langle \cdot, \cdot \rangle$ for the Euclidean inner product and $\| \cdot \|_2$ for the Euclidean norm. The unit sphere is $\mathbb{S}^{d-1}$. For $n \in \mathbb{N}$, let $[n] = \{1, \ldots, n\}$. For a (finite) set $\mathcal{S}$, $|\mathcal{S}|$ denotes its cardinality. We define $\mathrm{sign}(t) = +1$ if $t \geq 0$ and -1 otherwise. Unless stated otherwise, all random draws (parameters, demonstrations, query) are mutually independent. Finally, we write $\mathrm{softmax} : \mathbb{R}^K \to (0, 1)^K$ for the standard softmax with $K > 1$; for $\mathbf{z} = (z_1, \ldots, z_K)^\top \in \mathbb{R}^K$,

$$\mathrm{softmax}(\mathbf{z})_i \;=\; \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}\,.$$

2.1 Classification Tasks and Data Distributions

To investigate which function classes admit in-context learning in Transformers, prior work has focused largely on regression [25, 34, 27], with more recent studies on classification [31, 30]. To the best of our knowledge, Frei et al. [30] are among the first to study classification under a mixture-of-Gaussians model. While this stochastic formulation is natural, overlapping class-conditional distributions induce an irreducible error, which confounds the assessment of robustness: observed performance changes may reflect intrinsic label ambiguity rather than adversarial effects. For a cleaner evaluation, we use classification tasks that are clearly separable. Unlike Li et al. [31], we do not assume a task-shared, learnable discriminant direction, because zero-shot learning is not considered in this thesis. Building on these works, we first formalise a baseline linear classification problem.

Definition 2.1 (Linear classification task)**.** *Let $\mathbf{w}^\star$ be a random vector drawn uniformly from the d-dimensional unit sphere $\mathbb{S}^{d-1}$. Given $\mathbf{w}^\star$, an input–output pair $(\mathbf{x}, y)$ is generated as follows: for a feature vector $\mathbf{x} \sim \mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$, the score function is*

$$s(\mathbf{x}) = \langle \mathbf{w}^\star, \mathbf{x} \rangle$$

and the corresponding label is $y = \operatorname{sign}\big(s(\mathbf{x})\big)$.

This family induces linear decision boundaries. The left panel of Figure 2 illustrates the geometry for $d = 2$. Much existing work remains in the linear regime because it focuses on linear-attention-only Transformers. Kim et al. [56] analyse nonparametric regression rather than classification. Although Frei et al. [30] study a mixture-of-Gaussians setting with shared covariances and equal class priors, the Bayes-optimal decision boundary remains linear. Thus, since we compare linear and non-linear attention, it is natural to introduce a task with a non-linear decision boundary, which we use alongside the linear case to isolate the contribution of attention non-linearity in our robustness analysis.

Definition 2.2 (Quadratic classification task)**.** *Let $\boldsymbol{\theta}^\star = (\mathbf{A}^\star, \mathbf{b}^\star)$ be a random parameter drawn from a prior $P_{\boldsymbol{\Theta}}$ supported on symmetric matrices $\mathbf{A}^\star \in \mathbb{R}^{d\times d}$ and vectors $\mathbf{b}^\star \in \mathbb{R}^d$. Given $\boldsymbol{\theta}^\star$, an input–output pair $(\mathbf{x}, y)$ is generated as follows: for a feature vector $\mathbf{x} \sim \mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$, the score function is*

$$s(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}^\star \mathbf{x} + \langle \mathbf{b}^\star, \mathbf{x} \rangle$$

and the corresponding label is $y = \operatorname{sign}\big(s(\mathbf{x})\big)$.

Quadratic forms yield curved decision boundaries, but the class is broad. To bridge linear and non-linear tasks whilst keeping task difficulty tunable, we specialise to a structured subfamily in which $\mathbf{A}^\star$ and $\mathbf{b}^\star$ are determined by a single unit direction and a curvature parameter. This yields a parabolic decision boundary.

Definition 2.3 (Parabola classification task)**.** *Let $\mathbf{u}^\star$ be a random vector drawn uniformly from the unit sphere $\mathbb{S}^{d-1}$ and let $k \in \mathbb{R}_{>0}$ be a curvature parameter. Set $\mathbf{A}^\star = -k\big(\mathbf{I}_d - \mathbf{u}^\star \mathbf{u}^{\star\top}\big)$ and $\mathbf{b}^\star = \mathbf{u}^\star$. Given $(\mathbf{u}^\star, k)$, an input–output pair $(\mathbf{x}, y)$ is generated as follows: for a feature vector $\mathbf{x} \sim \mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$, the score function is*

$$s(\mathbf{x}) = -k\big(\|\mathbf{x}\|_2^2 - \langle \mathbf{u}^\star, \mathbf{x} \rangle^2\big) + \langle \mathbf{u}^\star, \mathbf{x} \rangle,$$

and the corresponding label is $y = \operatorname{sign}\big(s(\mathbf{x})\big)$.

The right panel of Figure 2 shows the case $d = 2$ with $k = 0.5$. To make the geometry explicit, consider an orthonormal basis $\{\mathbf{u}^\star, \mathbf{U}_\perp^\star\}$, where $\mathbf{U}_\perp^\star \in \mathbb{R}^{d\times(d-1)}$ contains orthonormal columns spanning the subspace orthogonal to $\mathbf{u}^\star$. Write $\mathbf{x} = \alpha\, \mathbf{u}^\star + \mathbf{U}_\perp^\star \boldsymbol{\beta}$, where $\alpha = \langle \mathbf{u}^\star, \mathbf{x} \rangle$ and $\boldsymbol{\beta} = \mathbf{U}_\perp^{\star\top} \mathbf{x} \in \mathbb{R}^{d-1}$. Substituting into the score yields

$$s(\mathbf{x}) = \alpha - k\|\boldsymbol{\beta}\|_2^2.$$

Hence the decision boundary is the paraboloid $\alpha = k\|\boldsymbol{\beta}\|_2^2$ opening along $\mathbf{u}^\star$. Constraining $k > 0$ restricts the positive region to the interior of the paraboloid. As $k \to 0^+$, the boundary approaches the hyperplane $\alpha = 0$, recovering the linear task in Definition 2.1 with $\mathbf{w}^\star = \mathbf{u}^\star$.

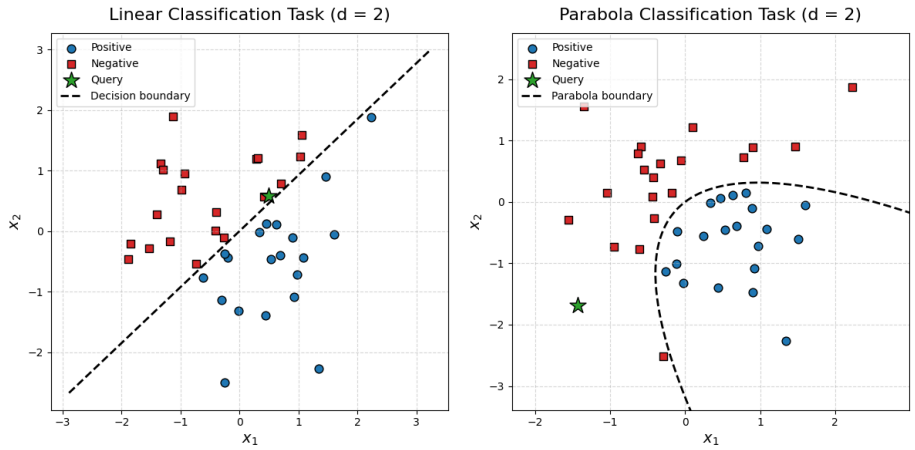


Figure 2: Linear ($d = 2$, left) and parabola ($d = 2$, right) classification tasks. Blue circles indicate positive examples, red squares negative examples, and the green star the query. The dashed line or curve shows the decision boundary. We set $k = 0.5$ in the parabola example.

To make explicit why k controls task difficulty, we compute the probability of a positive label. In the orthonormal basis $\{\mathbf{u}^\star, \mathbf{U}_\perp^\star\}$ above, write $\mathbf{x} = \alpha\,\mathbf{u}^\star + \mathbf{U}_\perp^\star\boldsymbol{\beta}$ with $\alpha = \langle \mathbf{u}^\star, \mathbf{x}\rangle$ and $\boldsymbol{\beta} = \mathbf{U}_\perp^{\star\top}\mathbf{x}$. Since $\mathbf{x} \sim \mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$ is isotropic, we have $\alpha \sim \mathcal{N}(0,1)$, $\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}_{d-1}, \mathbf{I}_{d-1})$ which are independent, hence $Z := \|\boldsymbol{\beta}\|_2^2 \sim \chi^2_{d-1}$, and Z is independent of α. Therefore

$$\mathbb{P}(y=+1) = \mathbb{P}\big(s(\mathbf{x})>0\big) = \mathbb{P}(\alpha > kZ) = \mathbb{E}_Z\big[\,1-\Phi(kZ)\,\big] = \int_0^\infty \big(1-\Phi(kz)\big)\, f_{\chi^2_{d-1}}(z)\,\mathrm{d}z,$$

where Φ is the standard normal cumulative distribution function and $f_{\chi^2_{d-1}}(z)$ is the χ^2_{d-1} probability density function. $\mathbb{P}(y = +1)$ decreases monotonically with k because $1 - \Phi(\cdot)$ is strictly decreasing in k. The more imbalanced label proportions indicate greater task curvature. We can use a standard Monte Carlo simulation to calculate an unbiased estimate of this probability [57].

Tokenisation. To pass these tasks to the Transformer as inputs for in-context learning, we now specify the tokenisation. The tokenisation below is tailored to encoder-style attention-only Transformers that use self-attention [58, 29].

Definition 2.4 (Prompt data matrix)**.** *Suppose* $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}, \mathbf{x}^{(\mathrm{query})}$ *are i.i.d. from* $\mathcal{P}_X$*, with corresponding true labels* $y^{(1)}, \ldots, y^{(n)}, y^{(\mathrm{query})}$*. Let* $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ *denote the set of demonstrations, and let* $\mathbf{x}^{(\mathrm{query})}$ *denote the query. The prompt data matrix is*

$$\mathbf{Z}_0 = \begin{bmatrix} \mathbf{x}^{(1)} & \cdots & \mathbf{x}^{(n)} & \mathbf{x}^{(\mathrm{query})} \\ y^{(1)} & \cdots & y^{(n)} & 0 \end{bmatrix} \in \mathbb{R}^{(d+1)\times(n+1)}.$$

In each prompt, the query occupies the last column and the n demonstrations are randomly permuted. No positional embeddings are used, ensuring invariance to demonstration order. The bottom entry of the query column is reserved for the label and is set to zero as a placeholder at prediction time. The model reads the columns of $\mathbf{Z}_0$ as a sequence of tokens

and, after a forward pass, takes the prediction from the bottom entry of the query column; the ground-truth label is $y^{(\text{query})}$. Other tokenisations for decoder-only Transformers also exist [25], but we do not discuss them in detail through this section.

2.2 Model Architecture

We study an attention-only Transformer to isolate the role of self-attention in ICL [26, 27, 33, 35]. In this architecture, we remove multi-head attention, multi-layer perceptron (MLP) blocks, and layer normalisation, while retaining residual connections at each layer. Let $P, Q \in \mathbb{R}^{(d+1)\times(d+1)}$ denote the value–projection and key–query transformation matrices, obtained from the standard formulation as $P := W_p W_v$ and $Q := W_k^\top W_q$. This yields a merged parameterisation of the attention core in the original Transformer [1]. The model consumes a prompt data matrix $\mathbf{Z}_0 \in \mathbb{R}^{(d+1)\times(n+1)}$ and applies two causal masks, $\mathbf{M}_0$ and $\mathbf{M}_{-\infty}$, both in $\mathbb{R}^{(n+1)\times(n+1)}$, so that the query neither contributes as a value nor can earlier tokens attend to it. The linear and softmax attention-only Transformers are defined separately below.

Definition 2.5 (L-layer linear attention-only Transformer)**.** *Given an input $\mathbf{Z}_0 \in \mathbb{R}^{(d+1)\times(n+1)}$ and layer parameters $\{(P_{\ell+1}, Q_{\ell+1})\}_{\ell=0}^{L-1}$, an L-layer linear attention-only Transformer updates at each layer by*

$$\mathbf{Z}_{\ell+1} \;=\; \mathbf{Z}_\ell \;+\; \frac{1}{n} P_{\ell+1}\, \mathbf{Z}_\ell\, \mathbf{M}_0 \left(\tfrac{1}{\sqrt{d}}\, \mathbf{Z}_\ell^\top Q_{\ell+1}\, \mathbf{Z}_\ell\right) \qquad \ell = 0, \ldots, L-1.$$

Definition 2.6 (L-layer softmax attention-only Transformer)**.** *With the same notation as above, an L-layer softmax attention-only Transformer updates at each layer by*

$$\mathbf{Z}_{\ell+1} \;=\; \mathbf{Z}_\ell \;+\; P_{\ell+1}\, \mathbf{Z}_\ell\, \mathbf{M}_0\, \mathrm{softmax}\Big(\mathbf{M}_{-\infty} + \tfrac{1}{\sqrt{d}}\, \mathbf{Z}_\ell^\top Q_{\ell+1}\, \mathbf{Z}_\ell\Big) \qquad \ell = 0, \ldots, L-1,$$

where $\mathrm{softmax}(\cdot)$ *is applied column-wise, so that each column of the weight matrix is non-negative and sums to one.*

Normalisation and masking. A factor $1/\sqrt{d}$ is included in both variants to control score magnitudes as the feature dimension grows [1]. In the linear case, the update is scaled by $1/n$ to average contributions across demonstration positions and to improve optimisation stability [58]. In the softmax case, $1/n$ is omitted because the column-wise softmax already normalises across positions.

Two masks are used with distinct roles, keeping notation consistent with the definitions above. The value mask $\mathbf{M}_0 := \begin{bmatrix} \mathbf{I}_n & \mathbf{0} \\ \mathbf{0}^\top & 0 \end{bmatrix} \in \mathbb{R}^{(n+1)\times(n+1)}$ zeroes the query column in the value term, so the query never contributes as a value. In the linear case, $\mathbf{M}_0$ is also applied to the attention score; since $\mathbf{M}_0$ is idempotent ($\mathbf{M}_0^2 = \mathbf{M}_0$), this is algebraically equivalent to a single right-multiplication by $\mathbf{M}_0$.

The second mask is used only in the softmax case. As attention weights are obtained by applying softmax to the score matrix, the last row cannot be removed by a simple matrix

product before normalisation. An additive mask $\mathbf{M}_{-\infty} \in \mathbb{R}^{(n+1)\times(n+1)}$ is therefore added to the unnormalised score matrix prior to the column-wise softmax, defined by

$$[\mathbf{M}_{-\infty}]_{ij} := \begin{cases} 0 & 1 \le i \le n \\ -\infty & i = n{+}1 \end{cases} \quad \text{for all } j \in \{1, \dots, n{+}1\}.$$

Thus the entire last row is set to $-\infty$, which drives the corresponding attention weights to zero, suppressing attention to the query from every position.

Readout. After L layers, the score for the query is read from the last row and last column of $\mathbf{Z}_L$ as

$$\widehat{s}\big(\mathbf{x}^{(\text{query})}\big) = \big[\mathbf{Z}_L\big]_{d+1,\,n+1}.$$

The predicted query label is then calculated as $\hat{y}^{(\text{query})} = \operatorname{sign}\Big(\widehat{s}\big(\mathbf{x}^{(\text{query})}\big)\Big)$. Some prior work adopts the convention $-[\mathbf{Z}_L]_{d+1,\,n+1}$ to facilitate algorithmic constructions within hand-designed Transformers for theory [26, 31]. This thesis retains the positive sign to match the embedding and labelling conventions.

Training objective. The model is trained with a squared-loss objective on the query score. Let $\mathcal{D}_{\text{train}}$ index the training tasks with cardinality $B_{\text{train}} := |\mathcal{D}_{\text{train}}|$. To make the model learn different tasks from the prompt on the fly, each training task $\tau \in \mathcal{D}_{\text{train}}$ contains n_{train} demonstrations and one query. The decision boundary is sampled independently for each task, ensuring that all tasks are distinct. The empirical loss is

$$\widehat{\mathcal{L}}(f) = \frac{1}{2B_{\text{train}}} \sum_{\tau \in \mathcal{D}_{\text{train}}} \Big(\widehat{s}_\tau(f) - y_\tau^{(\text{query})}\Big)^2,$$

where f denotes the model, $\widehat{s}_\tau(f)$ is the predicted score for the query and $y_\tau^{(\text{query})}$ is the true label in task τ. Although our task is binary classification, the squared loss is convex and differentiable, which facilitates analysis and stable optimisation in the ICL setting [31, 35, 26].

Testing objective. For model evaluation, we report query-level accuracy on separate test tasks. Let $\mathcal{D}_{\text{test}}$ index the test tasks with cardinality $B_{\text{test}} := |\mathcal{D}_{\text{test}}|$. Each task $\tau \in \mathcal{D}_{\text{test}}$ contains n_{test} demonstrations and a query, and tasks are also sampled independently from each other. To diagnose performance on both classes, we partition $\mathcal{T}_+ = \big\{\tau \in \mathcal{D}_{\text{test}} : y_\tau^{(\text{query})} = +1\big\}$, $\mathcal{T}_- = \big\{\tau \in \mathcal{D}_{\text{test}} : y_\tau^{(\text{query})} = -1\big\}$, with $B_{\text{test},+} := |\mathcal{T}_+|$ and $B_{\text{test},-} := |\mathcal{T}_-|$. The empirical positive and negative accuracies are

$$\widehat{A}^+(f; n_{\text{test}}) = \frac{1}{B_{\text{test},+}} \sum_{\tau \in \mathcal{T}_+} \mathbb{1}\big\{\widehat{s}_\tau(f) > 0\big\}, \quad \widehat{A}^-(f; n_{\text{test}}) = \frac{1}{B_{\text{test},-}} \sum_{\tau \in \mathcal{T}_-} \mathbb{1}\big\{\widehat{s}_\tau(f) < 0\big\},$$

where f denotes the model, $\widehat{s}_\tau(f)$ is the predicted query score on task τ and $\mathbb{1}\{\cdot\}$ is the indicator function. The empirical average accuracy reported is

$$\widehat{A}^{\text{avg}}(f; n_{\text{test}}) = \tfrac{1}{2}\Big(\widehat{A}^+(f; n_{\text{test}}) + \widehat{A}^-(f; n_{\text{test}})\Big).$$

To make the dependence on the test-time demonstration length explicit, we treat n_{test} as a fixed design parameter above.

Baseline Model (GPT-2). To provide a reference point, we include a small GPT-2–style decoder-only Transformer as a baseline. We do not elaborate on its formal definition here: it is used purely as a comparator in the robustness analyses to clarify the distinctions between the linear and softmax attention-only Transformers. The baseline follows a standard architecture with layer normalisation, multi-head causal self-attention and a position-wise feed-forward network [43]. Full configuration details are reported in Section 5. The implementation aligns with Garg et al. [25] in tokenisation and training procedure, and adapts their regression-style prompt to our classification setting by using labels in $\{\pm 1\}$.

3 Methodology

In this section, we consider two adversarial attacks, label flipping (LF) and label hijacking (LH), which reflect realistic deployment conditions, and subsequently defines the quantitative robustness metrics used for evaluation.

3.1 Adversarial Attacks

We introduce small-scale analogues of LLM jailbreaking and context hijacking in a binary classification setting. In line with these paradigms, attacks are introduced at test time, while model parameters remain frozen. Concretely, adversarial perturbations are applied to the test-time prompt data matrix, and the pre-trained Transformer is then evaluated on the perturbed prompts. Throughout this subsection, let n denote the number of demonstrations per prompt and N the number of attacked demonstrations, with $0 \leq N \leq n$. We present each attack in algorithmic form, following Wei et al. [21]. Recalling Definition 2.4, we denote a clean prompt data matrix by

$$\mathbf{Z}_0 = \begin{bmatrix} \mathbf{x}^{(1)} & \cdots & \mathbf{x}^{(n)} & \mathbf{x}^{(\text{query})} \\ y^{(1)} & \cdots & y^{(n)} & 0 \end{bmatrix} \in \mathbb{R}^{(d+1)\times(n+1)}.$$

Label flipping (LF) Prior work [22, 21] shows that inserting harmful or misaligned request–response pairs can compromise with LLM behaviour, leading to undesired outputs. In parallel, Zheng et al. [59] report that benign and harmful prompts form separable clusters in representation space. In our classification setting, the supervision signal naturally decomposes into two classes. A "harmful request" is modelled as label -1 (without loss of generality), indicating content that should be refused; flipping this label to $+1$ corresponds to permitting such content, that is, a semantically aligned shift from refusal to compliance. Mislabelling is therefore the most direct analogue of malicious interference in classification. Accordingly, labels of a subset of demonstrations are flipped so that the corrupted examples straddle the decision boundary. The following procedure generates a prompt data matrix with exactly N flipped labels.

Algorithm 3.1: Label flipping (LF)

1. Let $N \leq n$ be the number of flipped demonstrations.
2. Sample a subset $S \subseteq [n]$ with $|S| = N$ uniformly without replacement.
3. Flip the labels in S. Define the new labels as

$$\tilde{y}^{(i)} = \begin{cases} -\, y^{(i)} & i \in S, \\ ;\ y^{(i)} & i \notin S \end{cases}$$

4. Form the corrupted prompt data matrix

$$\mathbf{Z}_{\mathrm{LF},N}(S) = \begin{bmatrix} \mathbf{x}^{(1)} & \cdots & \mathbf{x}^{(n)} & \mathbf{x}^{(\mathrm{query})} \\ \tilde{y}^{(1)} & \cdots & \tilde{y}^{(n)} & 0 \end{bmatrix}.$$

With the query and all feature vectors unchanged, the attack operates purely in label space. Performance degradation is evaluated as a function of N. For each corruption level N, the subset S can be resampled independently across repetitions, providing estimation of mean accuracy change with confidence intervals. Frei et al. [30] also study label flipping, however, the perturbations are modelled as random label noise rather than an intentional adversary, thereby assessing non-adversarial robustness. By construction, the present attack enables analysis of how linear versus softmax attention allocate weight to contradictory evidence and adjust predictions when correct and incorrect demonstrations coexist. This, in turn, reveals architecture-specific failure modes and helps characterise the vulnerability of Transformer models to erroneous supervision data within the prompt.

Label hijacking (LH). Qiang et al. [23] construct adversarial prompts by learning confusing input strings from the model, for example by appending irrelevant suffixes. Unlike label flipping (LF), this attack does not inject incorrect answers. Instead, it modifies the inputs while the demonstration labels remain ostensibly correct. In our binary classification setting, such ambiguity corresponds to inputs that lie close to the decision boundary. Consequently, we perturb a subset of demonstrations from one class towards the boundary while preserving labels and leaving the other class unchanged. Before introducing the attack, we first formalise the notion of margin to specify how close points are moved to the boundary.

Definition 3.1 (Margin)**.** *Let $s : \mathbb{R}^d \to \mathbb{R}$ be a decision score and let the decision boundary be $\Gamma = \{\mathbf{x} \in \mathbb{R}^d : s(\mathbf{x}) = 0\}$. For $\delta \geq 0$, the margin of width δ is the tubular neighbourhood*

$$\mathcal{M}_\delta := \{\mathbf{x} \in \mathbb{R}^d : \operatorname{dist}(\mathbf{x}, \Gamma) \leq \delta\},$$

where $\operatorname{dist}(\cdot,\cdot)$ denotes the Euclidean distance. Equivalently, when Γ is smooth, $\mathcal{M}_\delta$ is the strip between the two offset sets $\Gamma_{\pm\delta} = \{\mathbf{z} \pm \delta\, \mathbf{z}_\perp : \mathbf{z} \in \Gamma\}$, where $\mathbf{z}_\perp$ is a unit vector normal to Γ at $\mathbf{z}$.

Now we formalise the procedure to implement the label hijacking attack.

Algorithm 3.2: Label hijacking (LH)

1. Let $s : \mathbb{R}^d \to \mathbb{R}$ be the task score with decision boundary $\Gamma = \{\mathbf{x} : s(\mathbf{x}) = 0\}$, and let $\mathcal{M}_\delta$ be the margin neighbourhood of width $\delta \geq 0$. Let $c \in \{-1, +1\}$ denote the query label, that is, $y^{(\text{query})} = c$. Define $I_c = \{\, i \in [n] : \, y^{(i)} = c \,\}$ as the indices of demonstrations in class c. Let $N \leq |I_c|$ be the number of hijacked demonstrations.
2. Sample a subset $S \subseteq I_c$ with $|S| = N$ uniformly without replacement.
3. Push the feature vectors for indices in S towards the decision boundary while preserving labels. Define the new feature vectors as

$$\tilde{\mathbf{x}}^{(i)} \;=\; \begin{cases} \mathbf{x}^{(i)} - c\, t_i \, \dfrac{\nabla s(\mathbf{x}^{(i)})}{\|\nabla s(\mathbf{x}^{(i)})\|_2} & \text{if } i \in S \\ \mathbf{x}^{(i)} & \text{if } i \notin S \end{cases}$$

   Here $t_i \geq 0$ is chosen so that $\operatorname{dist}\big(\tilde{\mathbf{x}}^{(i)}, \Gamma\big) \leq \delta$ and $\operatorname{sign}\big(s(\tilde{\mathbf{x}}^{(i)})\big) = c$. The gradient $\nabla s(\mathbf{x}^{(i)})$ is evaluated at $\mathbf{x}^{(i)}$.
4. Form the hijacked prompt data matrix

$$\mathbf{Z}_{\text{LH},N,\delta}(S) \;=\; \begin{bmatrix} \tilde{\mathbf{x}}^{(1)} & \cdots & \tilde{\mathbf{x}}^{(n)} & \mathbf{x}^{(\text{query})} \\ y^{(1)} & \cdots & y^{(n)} & 0 \end{bmatrix}.$$

Anwar et al. [29] generate hijacking contexts via gradient-based, targeted attacks that optimise a continuous loss and backpropagate to obtain adversarial inputs. In our binary setting, the objective is simply to shift the prediction from one class to the other. Unlike regression, there is no need to specify a target real-valued output. Hence, taking a geometry-driven approach, we select demonstrations that share the query label and move their feature vectors towards the decision boundary on the same side, within a prescribed margin, along the gradient of the task score ∇s. The resulting demonstrations near the boundary dilute the evidence in favour of the query class. This procedure is still gradient-based, but it relies on the gradient of the task score rather than model-specific gradients. It therefore avoids dependence on model architecture.

This construction complements label flipping. Labels remain fixed and the attack operates entirely on the feature side. During training, positive and negative examples are sampled across the feature space, yielding a balanced class signal. At test time, evidence for one class is weakened so that we can examine how the trained linear and softmax attention mechanisms reallocate weights under the altered data pattern. This evaluation also provides insights into whether the model has internalised the geometry of the decision rule.

3.2 Robustness metrics

Two levels of robustness metric are consider here: attack-specific and aggregate. We adopt the positive, negative, and balanced accuracies defined in Section 2. Throughout the robustness experiments, all attacks are evaluated under a fixed test-time demonstration budget n_{test}, so the number of demonstrations per task remains constant across corruption levels. For saving notation, we remove the explicit dependence on n_{test} and simply write $\widehat{A}^{+}(f)$, $\widehat{A}^{-}(f)$, and $\widehat{A}^{\text{avg}}(f)$. Unless stated otherwise, these refer to accuracies on a clean

test set.

Before stating the definitions, we introduce some additional notations. Let $\mathcal{R} = \{\mathrm{LF}, \mathrm{LH}\}$ denote the set of attack types. Let N be the corruption budget, namely the number of demonstrations attacked in a task. For a given $\rho \in \mathcal{R}$ and budget N, denote the attacked positive, negative and average accuracies by $\widehat{A}^{+}_{\rho,N}(f)$, $\widehat{A}^{-}_{\rho,N}(f)$ and $\widehat{A}^{\mathrm{avg}}_{\rho,N}(f)$, respectively.

Robustness score (RS). Following Laugros et al. [36], the robustness scores quantify the ratio of attacked to clean accuracies. Given an attack type ρ and a budget N, the empirical robustness scores are defined as

$$\mathrm{R}^{+}_{\rho,N}(f) := \frac{\widehat{A}^{+}_{\rho,N}(f)}{\widehat{A}^{+}(f)}, \quad \mathrm{R}^{-}_{\rho,N}(f) := \frac{\widehat{A}^{-}_{\rho,N}(f)}{\widehat{A}^{-}(f)}, \quad \mathrm{R}^{\mathrm{avg}}_{\rho,N}(f) := \frac{\widehat{A}^{\mathrm{avg}}_{\rho,N}(f)}{\widehat{A}^{\mathrm{avg}}(f)}.$$

Values close to 1 indicate stronger robustness, and $\mathrm{R}^{+}_{\rho,0}(f) = \mathrm{R}^{-}_{\rho,0}(f) = \mathrm{R}^{\mathrm{avg}}_{\rho,0}(f) = 1$ by construction. To summarise over a set of budgets $\mathcal{C}$, we can use

$$\mathrm{R}^{\bullet}_{\rho,\mathcal{C}}(f) := \frac{1}{|\mathcal{C}|} \sum_{N \in \mathcal{C}} \mathrm{R}^{\bullet}_{\rho,N}(f), \qquad \bullet \in \{+, -, \mathrm{avg}\}.$$

Mean Corruption Error (mCE). The robustness scores above are computed per attack type and per corruption level. Hendrycks and Dietterich [37] proposed an aggregate measure to compare models' overall robustness across all conditions, originating in computer vision and using AlexNet [60] as the reference model. In our setting, we make minor modifications and replace the baseline with a small GPT-2 model capable of performing ICL. Since the metric uses error, we first derive error from accuracy. Given ρ and N, and for $\bullet \in \{+, -, \mathrm{avg}\}$, we define error rates as

$$\widehat{E}^{\bullet}_{\rho,N}(f) := 1 - \widehat{A}^{\bullet}_{\rho,N}(f).$$

For each $\rho \in \mathcal{R}$, let $\mathcal{C}_\rho$ be the set of budgets considered for the attack. Let $f_{\text{GPT-2}}$ denote our baseline model. We define the mean corruption error (mCE) as

$$\mathrm{mCE}^{\bullet}(f) := \sum_{\rho \in \mathcal{R}} \sum_{N \in \mathcal{C}_\rho} \frac{\widehat{E}^{\bullet}_{\rho,N}(f)}{\widehat{E}^{\bullet}_{\rho,N}(f_{\text{GPT-2}})}.$$

Smaller values indicate greater robustness. The sets $\mathcal{R}$ and $\{\mathcal{C}_\rho\}_{\rho \in \mathcal{R}}$ are held fixed across models to ensure comparability.

4 One-Layer Attention-Only Transformers

In this section, we derive closed-form decision rules for our one-layer linear and softmax attention-only Transformers on the prompt data. A single forward pass in the linear architecture computes a linear decision rule at the query, whereas the softmax variant produces a normalised convex combination of value projections with data-dependent weights. We now state the two propositions.

Proposition 4.1 (One-layer linear forward pass)**.** *Given the prompt matrix* $\mathbf{Z}_0 = [\mathbf{z}_1 \cdots \mathbf{z}_n \, \mathbf{z}_q] \in \mathbb{R}^{(d+1)\times(n+1)}$ *with* $\mathbf{z}_j = \begin{bmatrix} \mathbf{x}^{(j)} \\ y^{(j)} \end{bmatrix}$ *for* $j \le n$ *and* $\mathbf{z}_q = \begin{bmatrix} \mathbf{x}^{(\text{query})} \\ 0 \end{bmatrix}$*. Let* $P, Q \in \mathbb{R}^{(d+1)\times(d+1)}$ *be the parameter matrices. The predicted query score after one forward pass of the one-layer linear attention-only Transformer (Definition 2.5 with* $L = 1$*) is*

$$\widehat{s}\big(\mathbf{x}^{(\text{query})}\big) \;=\; \widehat{\mathbf{w}}^{\top}\, \mathbf{x}^{(\text{query})}, \qquad \widehat{\mathbf{w}} \;=\; \frac{1}{n\sqrt{d}} \Big[\, Q^{\top}\mathbf{u} \,\Big]_{1:d},$$

where $\mathbf{u} := \sum_{j=1}^{n} \alpha_j \, \mathbf{z}_j$ *with* $\alpha_j := p^{\top}\mathbf{z}_j$*,* $p^{\top}$ *is the last row of* P *decomposed as* $p^{\top} = [\, p_x^{\top} \;\; p_y \,]$*, and* $[\cdot]_{1:d}$ *keeps the first* d *entries.*

Proof. Let $e_y \in \mathbb{R}^{d+1}$ and $e_q \in \mathbb{R}^{n+1}$ be the standard basis vectors selecting the last row and last column, respectively. Since $[\mathbf{Z}_0]_{d+1,n+1} = 0$ and $\mathbf{M}_0$ zeroes the query as a value, the query readout equals

$$\widehat{s} = \frac{1}{n\sqrt{d}}\, e_y^{\top} P\, \mathbf{Z}_0 \mathbf{M}_0 (\mathbf{Z}_0^{\top} Q \mathbf{Z}_0)\, e_q = \frac{1}{n\sqrt{d}} \sum_{j=1}^{n} \big(e_y^{\top} P \mathbf{z}_j\big)\, \big(\mathbf{z}_j^{\top} Q\, \mathbf{z}_q\big),$$

where the sum runs to n because the $(n{+}1)$-st column of $\mathbf{Z}_0\mathbf{M}_0$ is zero. Let $\alpha_j = e_y^{\top} P \mathbf{z}_j = p^{\top}\mathbf{z}_j$ and $\mathbf{u} = \sum_{j=1}^{n} \alpha_j \, \mathbf{z}_j$. With $\mathbf{z}_q = [\mathbf{x}^{(\text{query})}; 0]$,

$$\sum_{j=1}^{n} \alpha_j \, \mathbf{z}_j^{\top} Q \, \mathbf{z}_q = \mathbf{u}^{\top} Q \, \mathbf{z}_q = \big(Q^{\top}\mathbf{u}\big)_{1:d}^{\top} \mathbf{x}^{(\text{query})},$$

which gives $\widehat{s}(\mathbf{x}^{(\text{query})}) = \widehat{\mathbf{w}}^{\top}\mathbf{x}^{(\text{query})}$ with $\widehat{\mathbf{w}} = \frac{1}{n\sqrt{d}}[Q^{\top}\mathbf{u}]_{1:d}$. □

Proposition 4.2 (One-layer softmax forward pass)**.** *Given the prompt matrix* $\mathbf{Z}_0 = [\mathbf{z}_1 \cdots \mathbf{z}_n \, \mathbf{z}_q] \in \mathbb{R}^{(d+1)\times(n+1)}$ *with* $\mathbf{z}_j = \begin{bmatrix} \mathbf{x}^{(j)} \\ y^{(j)} \end{bmatrix}$ *for* $j \le n$ *and* $\mathbf{z}_q = \begin{bmatrix} \mathbf{x}^{(\text{query})} \\ 0 \end{bmatrix}$*. Let* $P, Q \in \mathbb{R}^{(d+1)\times(d+1)}$ *be the parameter matrices. The predicted query score after one forward pass of the one-layer softmax attention-only Transformer (Definition 2.6 with* $L = 1$*) is*

$$\widehat{s}\big(\mathbf{x}^{(\text{query})}\big) \;=\; \sum_{i=1}^{n} v_i \, \pi_{i,n+1},$$

where $v_i := [P\, \mathbf{z}_i]_{d+1}$ *and*

$$\pi_{i,n+1} \;=\; \frac{\exp\big(\langle \mathbf{z}_i,\, Q\, \mathbf{z}_q \rangle / \sqrt{d}\big)}{\sum_{k=1}^{n} \exp\big(\langle \mathbf{z}_k,\, Q\, \mathbf{z}_q \rangle / \sqrt{d}\big)} \quad \textit{for } i \in [n].$$

Proof. Define the column-wise attention matrix

$$\mathbf{\Pi} \;:=\; \operatorname{softmax}\Big(\mathbf{M}_{-\infty} + \tfrac{1}{\sqrt{d}}\, \mathbf{Z}_0^{\top} Q\, \mathbf{Z}_0\Big),$$

so the last row satisfies $\pi_{n+1,\cdot} = 0$. Since $[\mathbf{Z}_0]_{d+1,n+1} = 0$, the residual contributes nothing to the query score and

$$\widehat{s} = \big[\; P\, \mathbf{Z}_0\, \mathbf{M}_0\, \mathbf{\Pi} \;\big]_{d+1,n+1} = \sum_{i=1}^{n} \big[\; P\, \mathbf{z}_i \;\big]_{d+1} \pi_{i,n+1} = \sum_{i=1}^{n} v_i \, \pi_{i,n+1},$$

where the normalisation is column-wise, hence $\sum_{i=1}^{n} \pi_{i,n+1} = 1$ and $\pi_{i,n+1} \geq 0$. This yields the stated form. □

Interpretation. As we can see, a single linear-attention layer combines the demonstrations into an effective weight vector $\widehat{\mathbf{w}}$ that is independent of the query label, and it scores the query by the dot product $\widehat{\mathbf{w}}^\top \mathbf{x}^{(\text{query})}$. The softmax prediction rule depends on query-demonstration interactions in both the numerator and the denominator of the attention weights, $\pi_{i,n+1} = \exp(\langle \mathbf{z}_i, Q\mathbf{z}_q \rangle / \sqrt{d}) / \sum_{k=1}^{n} \exp(\langle \mathbf{z}_k, Q\mathbf{z}_q \rangle / \sqrt{d})$, which makes the readout more sensitive to changes in the data. Here $v_i = [P\,\mathbf{z}_i]_{d+1} = p^\top \mathbf{z}_i = p_x^\top \mathbf{x}^{(i)} + p_y\, y^{(i)}$ is the last coordinate of the value vector $P\mathbf{z}_i$, which can be viewed as the value projection onto the readout place. Consequently, $\widehat{s}(\mathbf{x}^{(\text{query})}) = \sum_{i=1}^{n} \pi_{i,n+1}\, v_i$ is a similarity-weighted convex combination of projected demonstration values. If a single demonstration is highly compatible with the query under Q, its weight $\pi_{i,n+1}$ can dominate the sum, causing the prediction to be strongly influenced by that demonstration.

5 Experiment

This section empirically evaluates two attention-only architectures on the linear and parabolic tasks, and examines their behaviour under adversarial prompts. The experiments are designed to characterise the mechanisms set out in Section 4 regarding how each model utilises demonstrations. We first describe the data-generation process and its alignment with prior work. We then show that the trained models perform in-context learning and generalise to fewer shots at test time. Next, we compare single-layer linear and softmax Transformers in isolation and analyse their robustness to adversarial attacks on both tasks. Finally, we evaluate robustness for all model variants under label flipping and label hijacking across a range of budgets, reporting the predefined metrics alongside figures and tables.

5.1 Data Preparation

We describe how the datasets are prepared for the two tasks. The feature dimension is fixed at $d = 20$ throughout, and we do not study dimensional effects. This setting is common in related work [31, 29]. We restrict the analysis to linear and parabolic classification, each with its own decision boundary. In the linear setting, $\mathbf{w}_\tau^\star$ is drawn independently for each task τ, thereby determining the separating hyperplane. In the parabolic setting, $\mathbf{u}^\star$ is drawn independently and a prescribed curvature parameter k specifies the boundary. We emphasise that, for $d = 20$, the curvature is fixed at $k = 0.02$ to control task difficulty. If $k > 0$ is too large, positive examples concentrate along the direction $\mathbf{u}^\star$ because a large projection onto $\mathbf{u}^\star$ is then required for a positive label, which leads to an imbalanced distribution of classes in feature space. This issue does not arise in the linear case, since under standard Gaussian features, symmetry yields $\mathbb{P}(y = +1) = \mathbb{P}(y = -1) = 0.5$. With $k = 0.02$, the probability of a positive label is approximately 0.35 in $d = 20$, as estimated by Monte Carlo simulation.

After fixing the decision boundary, we independently sample the demonstrations and the query for each task and compute their ground-truth labels. We fix $n_{\text{train}} = 60$. To avoid

confounding due to order and class imbalance in ICL [12, 13], we repeatedly resample until each task contains an *equal number* of positive and negative demonstrations, then *randomly permute* the demonstrations and append the query. The resulting data are assembled into a prompt data matrix.

Training Data. For the training set, we experimented with $5{,}000$ to $50{,}000$ training tasks and found that $30{,}000$ breaks the performance bottleneck and markedly improves ICL accuracy, with little additional gain beyond this scale. This choice is consistent with prior observations in linear regression, which indicate that, for $d \approx 16$, ICL emerges only after training on a dataset of size $|\mathcal{D}_{\text{train}}| \approx 2^{15}$ tasks [61]. We therefore *train on* $30{,}000$ *i.i.d. tasks* in both the linear and parabolic settings.

Validation and Testing Data. For validation and testing, we follow the same generation process as in training. We sample $3{,}000$ independent decision boundaries per family. To track accuracy separately for positive and negative queries, for each boundary we draw one positive and one negative query, then build two prompt matrices, yielding $6{,}000$ prompts per family. As validation is used for model selection, we set $n_{\text{val}} = n_{\text{train}} = 60$. At test time, to assess few-shot ICL, we reduce the number of demonstrations and evaluate across $n_{\text{test}} \in \{10, \ldots, 30\}$. In all cases, demonstrations are generated as before with balanced classes, randomly permuted, and the query appended as the last column of the prompt matrix.

5.2 Model Setup and ICL Results

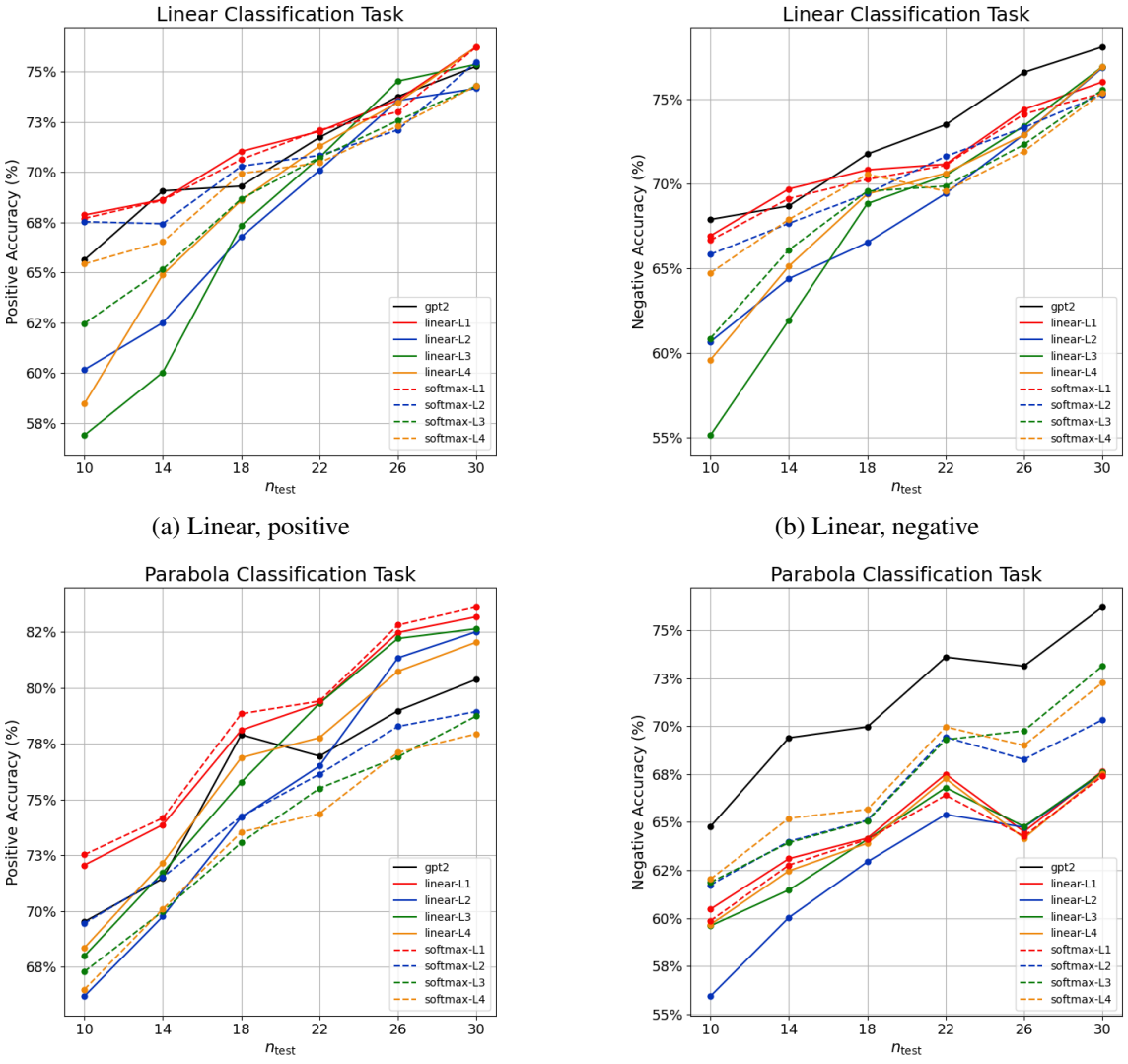
Table 1: Model configurations used in the experiments.

Model	Embedding size	#Layers	#Heads	#Parameters
Linear attention-only Transformer	21	1–4	1	0.9–3.5k
Softmax attention-only Transformer	21	1–4	1	0.9–3.5k
GPT–2	256	4	8	2.2M

We briefly summarise the models in Table 1. We train linear and softmax attention-only Transformers with $L \in \{1, 2, 3, 4\}$ layers on the two classification problems *separately*. The training objective is the squared loss on the query score defined in Section 2. We use stochastic optimisation with Adam [62] under a decaying learning-rate schedule, gradient clipping at 1.0, a batch size of 1024, and a total of $100{,}000$ update steps.

A small GPT–2 model is also trained as a baseline. Following Garg et al. [25], the prompt is tokenised from the interleaved data matrix. Under decoder-style causal masking, the model predicts sequentially, so we train it with a squared-loss objective averaged over all label positions. The GPT–2 configuration uses an embedding dimension of 256, 4 layers and 8 attention heads, and is trained with Adam under a decaying learning-rate schedule, gradient clipping at 2.0, a batch size of 256, and $50{,}000$ update steps. We evaluate on the validation set, recording positive and negative accuracies; model selection uses the checkpoint with the highest average validation accuracy, $\widehat{A}^{\text{avg}}_{\text{val}}$.

To present testing results, we assess generalisation to fewer demonstrations by varying the number of shots at test time, $n_{\text{test}} \in \{10, \ldots, 30\}$. We report accuracy curves for both task families in Figure 3. For clarity, the Transformers for the two task families are trained independently on their respective training sets: they share the same architecture but have different learned parameters.



(a) Linear, positive

(b) Linear, negative

(c) Parabola, positive

(d) Parabola, negative

Figure 3: Average accuracy on clean test data across demonstration counts $n_{\text{test}} \in \{10, 14, 18, 22, 26, 30\}$. Top row shows the linear task. Bottom row shows the parabola task. Left panels are on positive queries. Right panels are on negative queries.

We explicitly separate performance on positive and negative queries for both tasks. As expected, average ICL accuracy increases monotonically with the number of demonstrations, peaking at $n_{\text{test}} = 30$ [2, 25]. For the linear task, the class-wise accuracies are nearly identical (top row). In the few-shot regime, deeper linear models underperform deeper softmax models, and their performance converges as n_{test} increases. One-layer Transformers outperform deeper variants on the linear task, indicating that the task has

limited complexity and that additional depth tends to overfit.

By contrast, the parabola task shows clear differences across architectures. In the bottom row of Figure 3, the linear Transformer appears to overfit the positive class and performs poorly on negative queries relative to its softmax counterparts. This is consistent with our setting: positive examples concentrate along the direction $\mathbf{u}^\star$, and under the value mask both single- and multi-layer models implement a linear decision rule at the query. The resulting inductive bias is dominated by recovering $\mathbf{u}^\star$ during training. Notably, the one-layer softmax closely tracks the linear model, likely because a single layer is insufficient to realise the advantage of softmax attention for this function class. GPT–2 exhibits stable performance on both tasks.

Since all models perform well at $n_{\text{test}} = 30$, we fix the robustness evaluation at this demonstration count. Unless stated otherwise, all clean accuracies in the robustness experiments are computed based on the same data with demonstration length of $n_{\text{test}} = 30$.

5.3 Robustness of One-Layer Transformers

We first study one-layer attention-only Transformers and examine their robustness. To check whether the linear and softmax variants implement comparable decision rules, Figure 4 reports training loss and class-conditional validation accuracy.

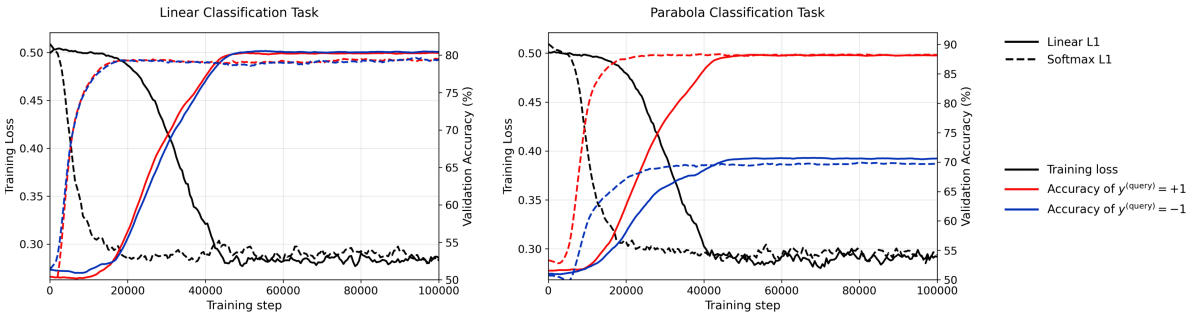
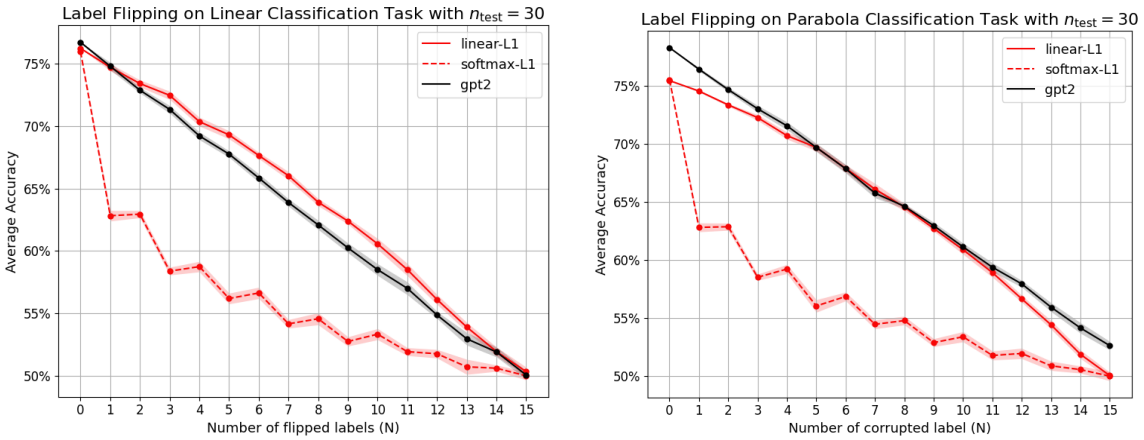


Figure 4: Training loss and validation accuracies for the one-layer linear and softmax attention-only Transformers. Solid and dashed lines denote the linear and softmax models, respectively. Black indicates training loss (left axis). Red and blue indicate validation accuracy for $y^{(\text{query})} = +1$ and $y^{(\text{query})} = -1$ (right axis).

On the linear task (left panel), both models quickly reach similar validation accuracy (about 80%) for $y^{(\text{query})} = \pm 1$, with the class-wise curves nearly overlapping. Differences in optimisation hyperparameters yield different convergence rates. On the parabola task (right panel), accuracy is higher for positive than for negative queries, and the two attention types continue to track each other closely during training. The training and validation results corroborate the evidence observed at test time. The single-layer linear and softmax models exhibit similar class-conditional behaviour on both tasks and therefore constitute a strong baseline for the robustness analysis that follows.

5.3.1 Robustness under Label Flipping (LF)

We implement label flipping at budgets $N \in \{1, \ldots, 15\}$ by uniformly selecting N demonstrations per prompt and inverting their labels. Figure 5 reports average accuracy as N increases.



(a) Linear task, LF

(b) Parabolic task, LF

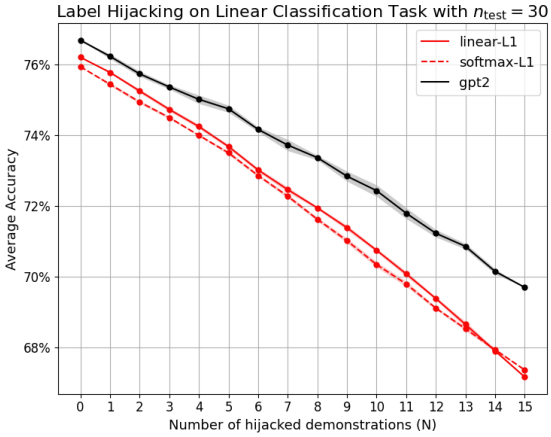
Figure 5: Label flipping (LF) robustness of one-layer Transformers with $n_{\text{test}} = 30$. Curves show average accuracy as the attack budget N increases. Solid and dashed lines denote linear and softmax attention respectively. The black curve is the result of GPT–2.

On both the linear and parabola tasks, the softmax variant exhibits a sharp initial decrease of about 13% points at $N{=}1$. Thereafter, its accuracy declines more slowly and approximately linearly. By contrast, the linear variant decreases at an almost constant slope across budgets, from 76% to 50% by $N{=}15$. At $N{=}15$, where half the demonstrations are corrupted, all models approach the accuracy of *random guessing*. The GPT–2 baseline tracks the one-layer linear variant closely, suggesting that a small model suffices for this setting. Despite learning comparable rules on clean data, the one-layer softmax Transformer degrades disproportionately under boundary-shifted demonstrations. A single softmax layer therefore provides no robustness advantage over the linear baseline.

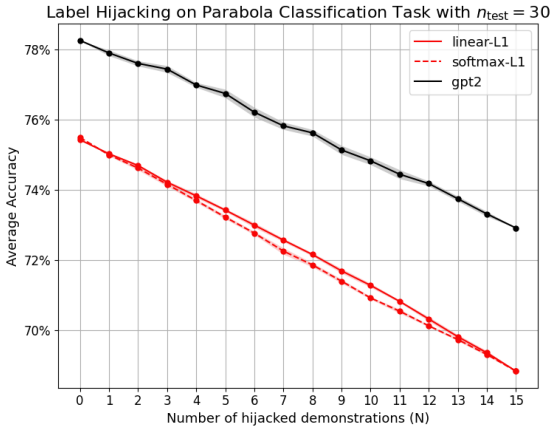
5.3.2 Robustness under Label Hijacking (LH)

We next assess robustness under label hijacking by moving $N \in \{1, \ldots, 15\}$ selected demonstrations towards the decision boundary within a margin neighbourhood of width $\delta = 0.1$ while keeping their labels unchanged. Figure 6 reports the results.

Unlike in the case of label flipping, on both the linear and parabolic tasks the one-layer linear and softmax models decline steadily (from 76% to 67% and from 75.5% to 69%, respectively). Consistent with the clean-test evidence and Figure 4, LH maintains the near-equivalence of the one-layer variants, indicating that at this depth robustness is driven more by task geometry than by the choice of attention mechanism for single layer Transformers.



(a) Linear task, LH



(b) Parabola task, LH

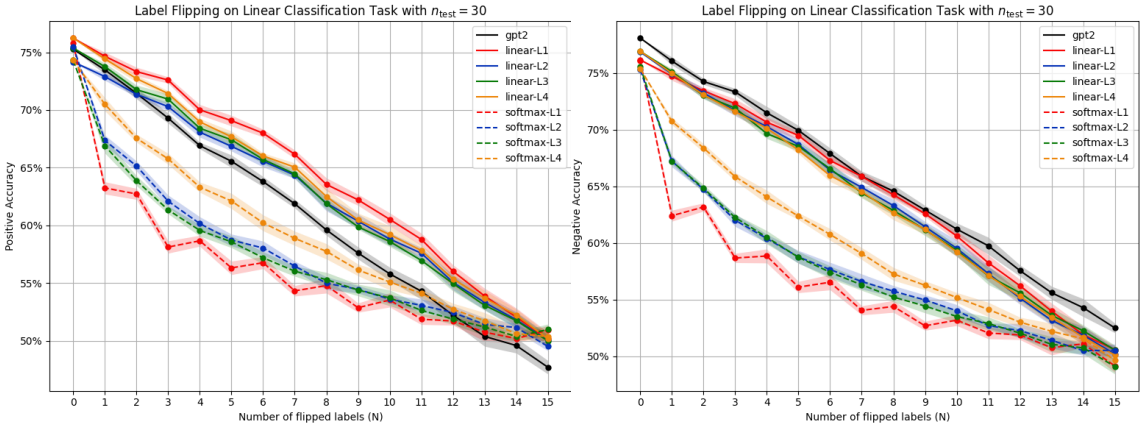
Figure 6: Label hijacking (LH) robustness of one-layer Transformers with $n_{\text{test}} = 30$. Curves show average accuracy versus the number of hijacked demonstrations N. Solid and dashed lines denote linear and softmax attention, respectively. The black curve is the result of GPT–2.

5.4 Robustness Evaluation

We synthesise robustness across all trained Transformers and examine the effect of depth. Figures 7 (LF) and 8 (LH) report positive and negative accuracies for the linear and parabolic tasks, with 95% confidence intervals, as the corruption budget N increases. Tables 2 and 3 summarise performance by architecture and depth, enabling direct comparison via robustness scores and mCE.

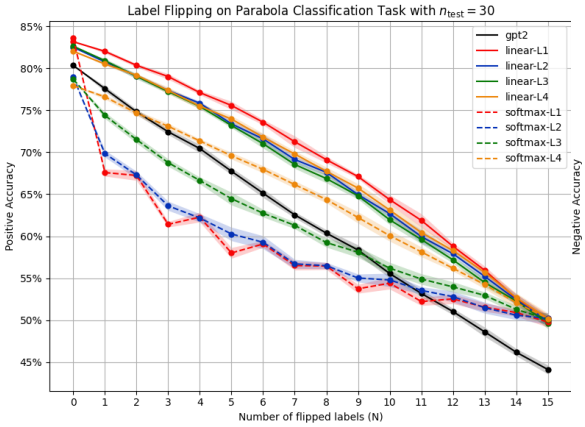
Figures 7 and 8. Under label flipping (Fig. 7), accuracies decrease monotonically with the budget N for all models. On the linear task, linear attention consistently outperforms softmax at matched depth. Also, adding layers primarily narrows the gap but does not change this ordering. On the parabolic task, deeper softmax models approach the linear counterparts and the positive–negative gap narrows, indicating reduced class-conditional inductive bias. Under label hijacking (Fig. 8), degradation is mild. Softmax exhibits a clear depth benefit on the parabolic task: for $L \geq 3$ it largely maintains both positive and negative accuracies, whereas linear attention degrades for both; on the linear task, linear attention remains slightly superior across depths.

In conclusion, label flipping reveals a persistent vulnerability of softmax to label corruption. By contrast, on curved decision boundaries label hijacking favours deeper softmax, yielding more balanced class-conditional predictions. Depth helps, but its gains are geometry-dependent: it stabilises softmax under label hijacking on the parabolic family, yet does not close the label flipping gap on the linear family.

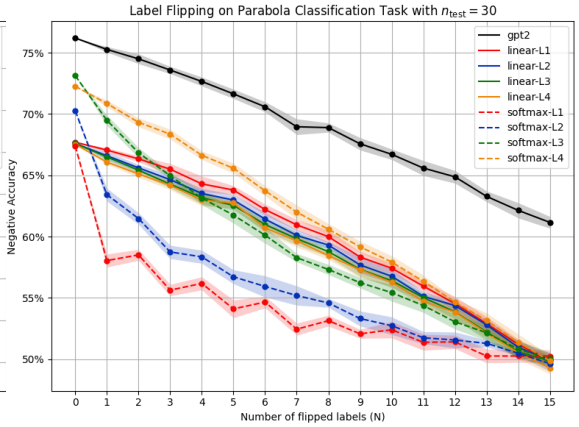


(a) Linear, positive

(b) Linear, negative

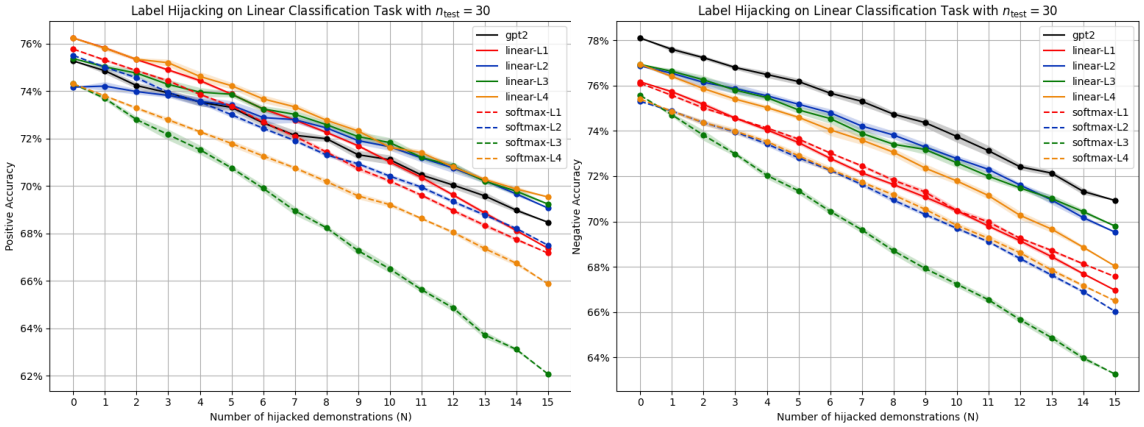


(c) Parabola, positive



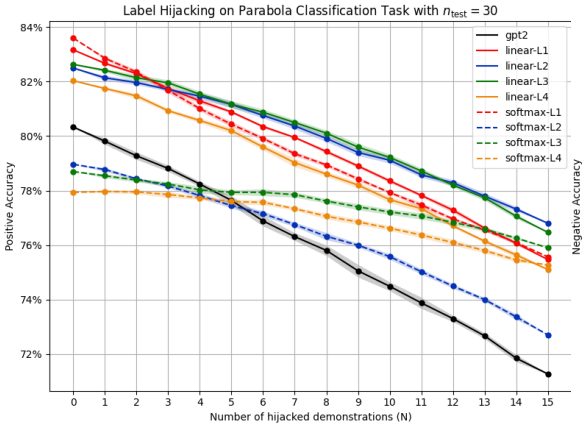
(d) Parabola, negative

Figure 7: Label flipping (LF) robustness across all models with $n_{\text{test}} = 30$. Top row: linear task; bottom row: parabolic task. Columns: positive and negative query accuracy. Solid and dashed lines denote linear and softmax attention; the black curve is GPT–2.

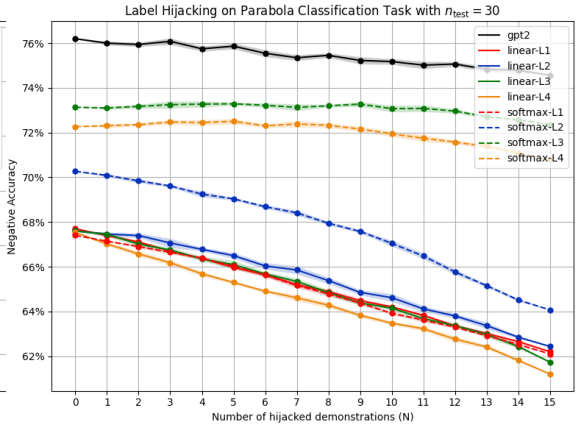


(a) Linear, positive

(b) Linear, negative



(c) Parabola, positive



(d) Parabola, negative

Figure 8: Label hijacking (LH) robustness across all models with $n_{\text{test}} = 30$. Top row: linear task; bottom row: parabolic task. Columns: positive and negative query accuracy. Solid and dashed lines denote linear and softmax attention; the black curve is GPT–2.

Robustness Score (RS) We compute robustness scores across all depths and budgets $C = \{1, \ldots, 15\}$. The summaries under LF and LH are given in Table 2.

Table 2: Robustness scores (average) under label flipping and label hijacking at $n_{\text{test}} = 30$.

Task	Attack	Linear				Softmax				GPT-2
		L1	L2	L3	L4	L1	L2	L3	L4	
Linear	Label flipping	0.832	0.826	0.819	0.817	0.725	0.751	0.752	0.783	0.811
	Label hijacking	0.942	0.964	0.957	0.949	0.942	0.943	0.914	0.942	0.954
Parabola	Label flipping	0.844	0.835	0.829	0.836	0.730	0.755	0.782	0.828	0.824
	Label hijacking	0.955	0.966	0.963	0.955	0.952	0.963	0.991	0.991	0.965

At $n_{\text{test}} = 30$, class-averaged robustness exhibits clear geometry dependence. Under LF, linear attention outperforms softmax at every depth on both tasks. Softmax attention

improves with depth, narrowing the gap on the parabolic task, while the gap persists on the linear task. The GPT–2 reference lies between them. Under LH, all models remain highly robust. On the linear task, linear attention is marginally ahead of softmax across depths. On the parabolic task, deeper softmax achieves the highest scores and slightly surpasses linear.

Mean Corruption Error (mCE). The mean corruption error summarises performance over both attack types and all budgets, normalised to the GPT–2 reference, and is therefore suited to comparing architectures across depths. Results are reported in Table 3. On the linear task, linear attention attains lower mCE than softmax at every depth, and depth has only a modest effect on the linear variant. On the parabolic task, softmax benefits from additional layers and ultimately surpasses linear, whereas the linear variant degrades slightly with depth.

Table 3: Mean corruption error (mCE) (divided by 30) for all Transformer variants on the linear and parabolic tasks at $n_{\text{test}} = 30$.

Task	**Linear**				**Softmax**			
	L1	**L2**	**L3**	**L4**	**L1**	**L2**	**L3**	**L4**
Linear	1.009	1.006	1.003	1.004	1.139	1.123	1.175	1.103
Parabolic	1.083	1.090	1.098	1.114	1.223	1.204	1.086	1.058

6 Conclusions

This study isolates the roles of *softmax* and *linear* attention in attention-only Transformers for binary classification and examines adversarial robustness under controlled prompts. Unlike prior work on large models [20, 19] or highly stylised settings [29, 31], we introduce a new family of data distributions and extend the linear-attention framework to a matched softmax counterpart with unified masking, readout and normalisation. We design simple, geometry-aware prompt attacks and adapt *robustness score* and *mean corruption error* to ICL as concise, class-conditional reporting tools [36].

Under *label flipping*, accuracy decreases monotonically and the linear model is consistently more robust than the softmax model, which is sensitive to single misleading demonstrations. Under *label hijacking*, one-layer variants behave similarly with mild degradation. With increasing depth, the softmax architecture becomes more robust on the parabolic task, while linear attention remains slightly superior on the linear task. Overall, prompt-time robustness is governed by the attention rule, task geometry and depth. Our controlled design isolates when differences arise from non-linear attention rather than from other factors.

The analysis is experimental and does not provide a theoretical account for deeper stacks. The attacks and geometries are simplified and may leave blind spots. We do not identify which demonstrations exert the greatest influence on each mechanism. The models are small attention-only stacks without multi-head attention, positional encodings or feed-forward blocks, so external validity to modern LLMs is indirect.

Future studies should move toward realistic deployment. Softmax attention should be examined under richer data, including non-parametric regression [56]. Scaling to multi-head and deeper architectures with positional structure is needed. Threat models should be broadened to include adaptive, targeted, ordering and insertion attacks, together with combined label–feature perturbations. Robustness under out-of-distribution shifts should be assessed [25]. Finally, these controlled insights should be bridged to large models through mechanistic analyses [20, 19].

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology*, 16(5):1–72, 2025.

[4] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

[5] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.

[6] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[7] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. Acecoder: Utilizing existing code to enhance code generation. *arXiv preprint arXiv:2303.17780*, 2023.

[8] Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. Want to reduce labeling cost? gpt-3 can help. *arXiv preprint arXiv:2108.13487*, 2021.

[9] Bosheng Ding, Chengwei Qin, Linlin Liu, Yew Ken Chia, Shafiq Joty, Boyang Li, and Lidong Bing. Is gpt-3 a good data annotator? *arXiv preprint arXiv:2212.10450*, 2022.

[10] Seongjin Shin, Sang-Woo Lee, Hwijeen Ahn, Sungdong Kim, HyoungSeok Kim, Boseop Kim, Kyunghyun Cho, Gichang Lee, Woomyoung Park, Jung-Woo Ha, et al.

On the effect of pretraining corpora on in-context learning by a large-scale language model. *arXiv preprint arXiv:2204.13509*, 2022.

[11] Steve Yadlowsky, Lyric Doshi, and Nilesh Tripuraneni. Pretraining data mixtures enable narrow model selection capabilities in transformer models. *arXiv preprint arXiv:2311.00871*, 2023.

[12] Stephanie Chan, Adam Santoro, Andrew Lampinen, Jane Wang, Aaditya Singh, Pierre Richemond, James McClelland, and Felix Hill. Data distributional properties drive emergent in-context learning in transformers. *Advances in neural information processing systems*, 35:18878–18891, 2022.

[13] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *International conference on machine learning*, pages 12697–12706. PMLR, 2021.

[14] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint arXiv:2202.12837*, 2022.

[15] Ekin Akyürek, Bailin Wang, Yoon Kim, and Jacob Andreas. In-context language learning: Architectures and algorithms. *arXiv preprint arXiv:2401.12973*, 2024.

[16] Boxin Wang, Weixin Chen, Hengzhi Pei, Chulin Xie, Mintong Kang, Chenhui Zhang, Chejian Xu, Zidi Xiong, Ritik Dutta, Rylan Schaeffer, et al. Decodingtrust: A comprehensive assessment of trustworthiness in gpt models. In *NeurIPS*, 2023.

[17] Houssem Ben Braiek and Foutse Khomh. Machine learning robustness: A primer. In *Trustworthy AI in Medical Imaging*, pages 37–71. Elsevier, 2025.

[18] Timo Freiesleben and Thomas Grote. Beyond generalization: a theory of robustness in machine learning. *Synthese*, 202(4):109, 2023.

[19] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[20] Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, and Yang Liu. Pandora: Jailbreak gpts by retrieval augmented generation poisoning. *arXiv preprint arXiv:2402.08416*, 2024.

[21] Zeming Wei, Yifei Wang, Ang Li, Yichuan Mo, and Yisen Wang. Jailbreak and guard aligned language models with only few in-context demonstrations. *arXiv preprint arXiv:2310.06387*, 2023.

[22] Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, et al. Many-shot jailbreaking. *Advances in Neural Information Processing Systems*, 37:129696–129742, 2024.

[23] Yao Qiang, Xiangyu Zhou, and Dongxiao Zhu. Hijacking large language models via adversarial in-context learning. *arXiv preprint arXiv:2311.09948*, 2023.

[24] Jiongxiao Wang, Zichen Liu, Keun Hee Park, Zhuojun Jiang, Zhaoheng Zheng, Zhuofeng Wu, Muhao Chen, and Chaowei Xiao. Adversarial demonstration attacks on large language models. *arXiv preprint arXiv:2305.14950*, 2023.

[25] Shivam Garg, Dimitris Tsipras, Percy S Liang, and Gregory Valiant. What can transformers learn in-context? a case study of simple function classes. *Advances in Neural Information Processing Systems*, 35:30583–30598, 2022.

[26] Johannes Von Oswald, Eyvind Niklasson, Ettore Randazzo, Joao Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 35151–35174. PMLR, 23–29 Jul 2023.

[27] Kwangjun Ahn, Xiang Cheng, Hadi Daneshmand, and Suvrit Sra. Transformers learn to implement preconditioned gradient descent for in-context learning. *Advances in Neural Information Processing Systems*, 36:45614–45650, 2023.

[28] Gen Li, Yuchen Jiao, Yu Huang, Yuting Wei, and Yuxin Chen. Transformers meet in-context learning: A universal approximation theory. *arXiv preprint arXiv:2506.05200*, 2025.

[29] Usman Anwar, Johannes Von Oswald, Louis Kirsch, David Krueger, and Spencer Frei. Adversarial robustness of in-context learning in transformers for linear regression, 2024.

[30] Spencer Frei and Gal Vardi. Trained transformer classifiers generalize and exhibit benign overfitting in-context. *arXiv preprint arXiv:2410.01774*, 2024.

[31] Tianle Li, Chenyang Zhang, Xingwu Chen, Yuan Cao, and Difan Zou. On the robustness of transformers against context hijacking for linear classification. *arXiv preprint arXiv:2502.15609*, 2025.

[32] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[33] Ruiqi Zhang, Spencer Frei, and Peter L Bartlett. Trained transformers learn linear models in-context. *arXiv preprint arXiv:2306.09927*, 2023.

[34] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? investigations with linear models. *arXiv preprint arXiv:2211.15661*, 2022.

[35] Yu Huang, Yuan Cheng, and Yingbin Liang. In-context convergence of transformers. *arXiv preprint arXiv:2310.05249*, 2023.

[36] Alfred Laugros, Alice Caplier, and Matthieu Ospici. Are adversarial robustness and common perturbation robustness independant attributes? In *Proceedings of the IEEE/CVF international conference on computer vision workshops*, pages 0–0, 2019.

[37] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. *arXiv preprint arXiv:1903.12261*, 2019.

[38] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(9):5149–5169, 2021.

[39] Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997.

[40] Yu Zhang and Qiang Yang. An overview of multi-task learning. *National Science Review*, 5(1):30–43, 2018.

[41] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

[42] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[43] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[44] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*, 22(10):1345–1359, 2009.

[45] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[46] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080*, 2021.

[47] Xinyi Wang, Wanrong Zhu, Michael Saxon, Mark Steyvers, and William Yang Wang. Large language models are latent variable models: Explaining and finding good demonstrations for in-context learning. *Advances in Neural Information Processing Systems*, 36:15614–15638, 2023.

[48] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.

[49] Yu Bai, Fan Chen, Huan Wang, Caiming Xiong, and Song Mei. Transformers as statisticians: Provable in-context learning with in-context algorithm selection. *Advances in neural information processing systems*, 36, 2024.

[50] Tom B Brown, Dandelion Mané, Aurko Roy, Martín Abadi, and Justin Gilmer. Adversarial patch. *arXiv preprint arXiv:1712.09665*, 2017.

[51] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.

[52] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Šrndić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. Evasion attacks against machine learning at test time. In *Joint European conference on machine learning and knowledge discovery in databases*, pages 387–402. Springer, 2013.

[53] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 ieee symposium on security and privacy (sp)*, pages 39–57. Ieee, 2017.

[54] Battista Biggio, Blaine Nelson, and Pavel Laskov. Poisoning attacks against support vector machines. *arXiv preprint arXiv:1206.6389*, 2012.

[55] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. Jailbreak attacks and defenses against large language models: A survey. *arXiv preprint arXiv:2407.04295*, 2024.

[56] Juno Kim, Tai Nakamaki, and Taiji Suzuki. Transformers are minimax optimal nonparametric in-context learners. *Advances in Neural Information Processing Systems*, 37:106667–106713, 2024.

[57] Samik Raychaudhuri. Introduction to monte carlo simulation. In *2008 Winter simulation conference*, pages 91–100. IEEE, 2008.

[58] Ruiqi Zhang, Spencer Frei, and Peter L Bartlett. Trained transformers learn linear models in-context. *Journal of Machine Learning Research*, 25(49):1–55, 2024.

[59] Chujie Zheng, Fan Yin, Hao Zhou, Fandong Meng, Jie Zhou, Kai-Wei Chang, Minlie Huang, and Nanyun Peng. On prompt-driven safeguarding for large language models. *arXiv preprint arXiv:2401.18018*, 2024.

[60] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[61] Allan Raventós, Mansheej Paul, Feng Chen, and Surya Ganguli. Pretraining task diversity and the emergence of non-bayesian in-context learning for regression. *Advances in neural information processing systems*, 36:14228–14246, 2023.

[62] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.